

MSP430 Hardware Multiplier Accelerator

MPY, MPYS, MAC and MACS over a 16-bit bus

Bibek Shrestha | September 2026 | SystemVerilog, VCS, Design Compiler, SAED 90 nm

The assignment

Design and test a hardware multiplier for the MSP430 that supports unsigned (MPY) and signed (MPYS) multiply, and unsigned (MAC) and signed (MACS) multiply-accumulate. The CPU picks the mode and first operand by writing to 0x0130 / 0x0132 / 0x0134 / 0x0136. Writing the second operand to 0x0138 starts the operation. The 32-bit result is read at 0x013A (low 16 bits) and 0x013C (high 16 bits). Ports: clk, reset_n (active low), 16-bit addr, 16-bit bidirectional data_bus, write_enable, read_enable, and output busy. Write a testbench that verifies the design thoroughly.

At a glance

Method	16-cycle shift-and-add on magnitudes, sign fixed at the end
Flip-flops / cells	153 / 881
Area (cell / total)	12677.53 / 13462.00
Clock	7 ns (≈ 143 MHz), slack 0.00 ns MET
Critical path	mcand_reg[0] $\rightarrow$ 32-bit adder $\rightarrow$ product_reg[31]
Busy time	18 clocks per operation
Tests	228 (RTL) and 228 (gate level), 0 errors
Tri-state check	0 BSLEX cells, 16 TNBUFFX cells

1 The four modes

All four multiply two 16-bit numbers into a 32-bit result. They differ in two ways: are the numbers signed, and does the new product replace the result or add to it?

Mode	op1 address	Numbers	Result	mode[1:0]
MPY	0x0130	unsigned	result = $a \times b$	00
MPYS	0x0132	signed	result = $a \times b$	01
MAC	0x0134	unsigned	result = result + $a \times b$	10
MACS	0x0136	signed	result = result + $a \times b$	11
OP2	0x0138	write op2 and start		
RESL / RESH	0x013A / 0x013C	read result[15:0] / result[31:16]		

Table 1: Register map. Mode bit 0 = signed, bit 1 = accumulate (it equals address bits [2:1]).

Example of why signed matters: 0xFFFF is 65535 unsigned but -1 signed. MPY gives $65535 \times 65535 = 0xFFFE0001$; MPYS gives $(-1) \times (-1) = 0x00000001$.

2 Design

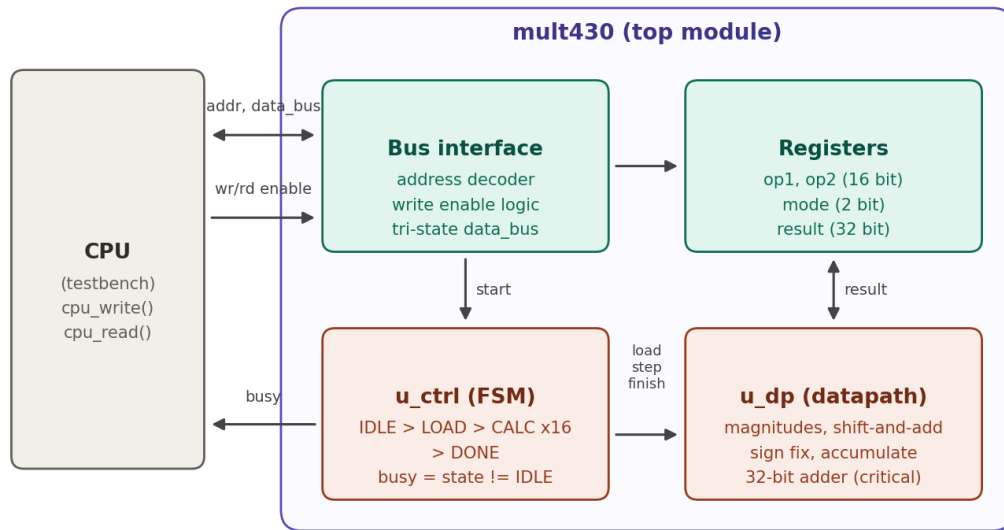


Figure 1: Architecture: a bus interface and registers on the CPU side, a control FSM and a datapath on the compute side.

Module	Job	Registers
mult430 (top)	Address decoder, op1/op2/mode registers, read mux, tri-state data_bus	34
mult430_ctrl	FSM and step counter, drives busy	6
mult430_dp	Magnitudes, shift-and-add, sign fix, accumulate	113

Table 2: Three modules, 153 flip-flops in total.

How one multiplier handles all four modes

1. **Signed modes:** make negative operands positive ($|x| = \sim x + 1$) and remember if the signs differ.
2. **Multiply** the two positive numbers with shift-and-add, one bit per clock, 16 clocks.
3. **Fix the sign:** if the signs differed, negate the product.
4. **Accumulate modes:** add the product to the old result instead of replacing it.

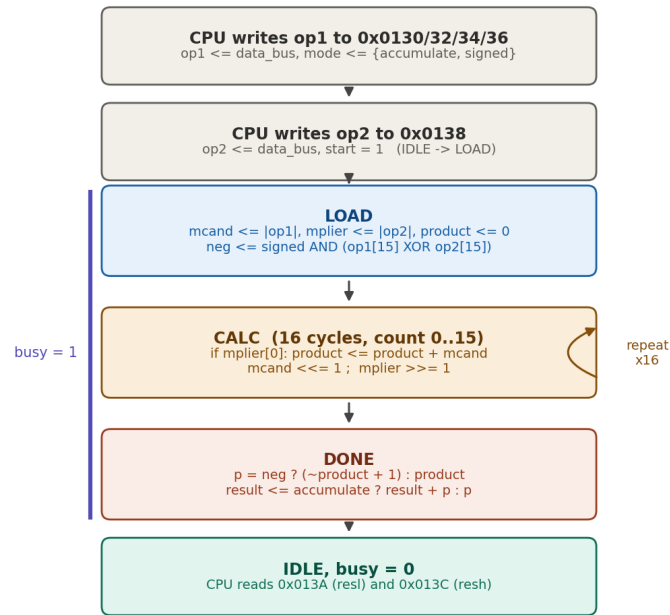


Figure 2: One operation from start to finish.

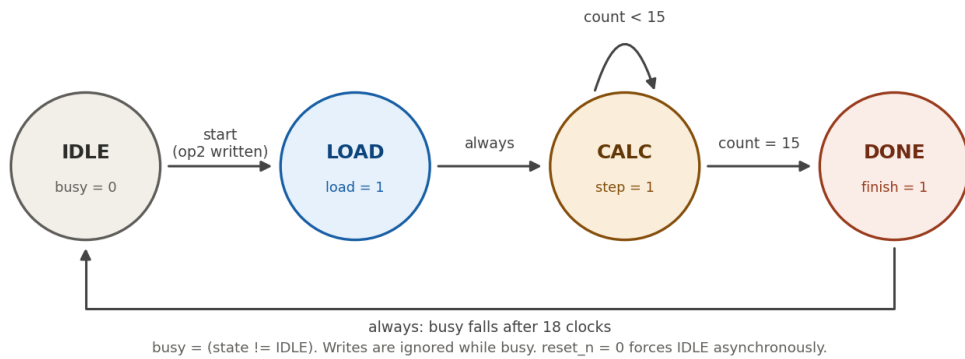


Figure 3: Control FSM. busy is high in every state except IDLE: 1 + 16 + 1 = 18 clocks.

Worked example: MPY 3×5

Step	multiplier bit	Action	product	multiplicand after
1	1	add 3	3	6
2	0	skip	3	12
3	1	add 12	15	24
4 to 16	0	skip	15	...

Table 3: Shift-and-add. For MPYS -3×5 the same steps give 15, then negation gives $0xFFFFFFFF1 = -15$.

The shared bus

The data bus is used by both sides, so only one may drive it at a time. The multiplier drives it only during a read of 0x013A or 0x013C; otherwise it lets go (z). Writes are ignored while busy is high.

```
assign wr_ok = write_enable && !read_enable && !busy;
```

```
assign start      = wr_ok && (addr == ADDR_OP2);
assign data_bus = rd_en ? rd_data : 16'bz; // tri-state driver
```

3 Testbench

The testbench acts as the CPU. `cpu_write` and `cpu_read` perform bus cycles, a small software model computes the expected answer, and every result is checked automatically. Inputs change on the falling clock edge to avoid races.

Group	Tests	What it checks	
MPY / MPYS / MAC / MACS	11	Each mode, largest values, -32768×-32768 , accumulation up and down	PASS
Write while busy	1	A MAC write during an MPY is ignored (0x31, not 0x72)	PASS
Bus released	2	<code>data_bus</code> is z when idle and on a non-result read	PASS
Mode switching	5	The mode never “sticks” from the previous operation	PASS
Overflow	5	The 32-bit accumulator wraps around both ways	PASS
Reset mid-operation	4	busy drops instantly, result cleared, then works again	PASS
Random	200	All modes, seed 430, biased to 0000/FFFF/8000/7FFF	PASS
Total	228		0 errors

Table 4: Test plan. The random split was MPY 51, MPYS 48, MAC 53, MACS 48.

Operation	Product	Result
MPYS 0×-1	0	0x00000000
MAC 2×10	+20	0x00000014
MAC 4×10	+40	0x0000003C
MACS -2×10	-20	0x00000028
MACS -5×-5	+25	0x00000041

Table 5: MAC and MACS keep a running total in the same result register.

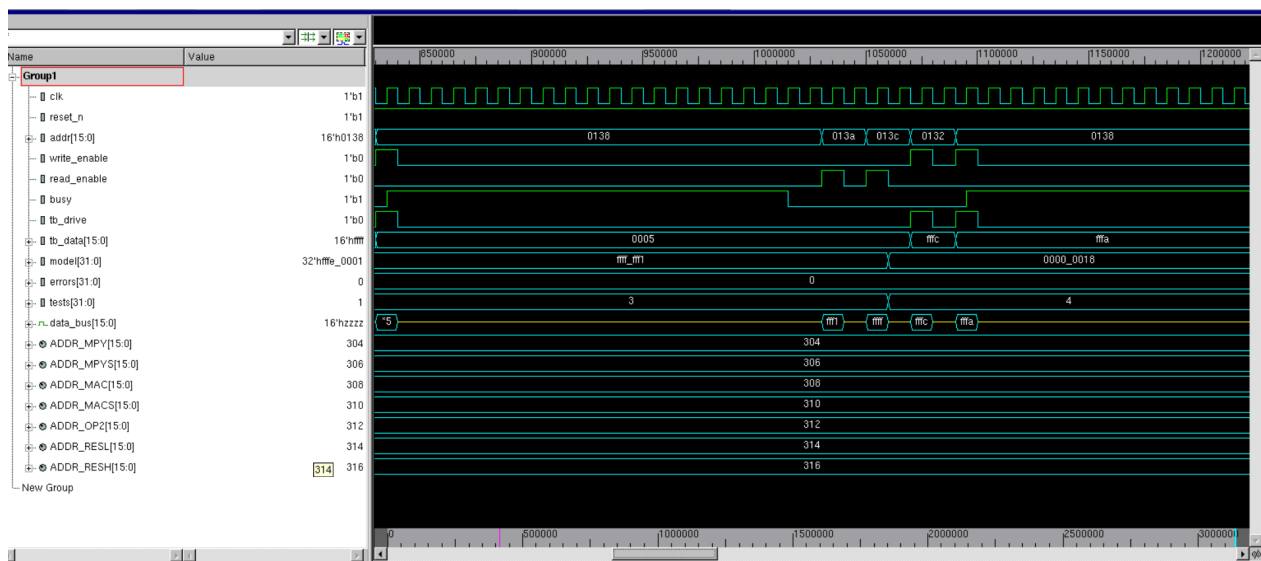


Figure 4: MPYS -3×5 : busy goes high after the op2 write; the two reads return fff1 and ffff = 0xFFFFFFFF.

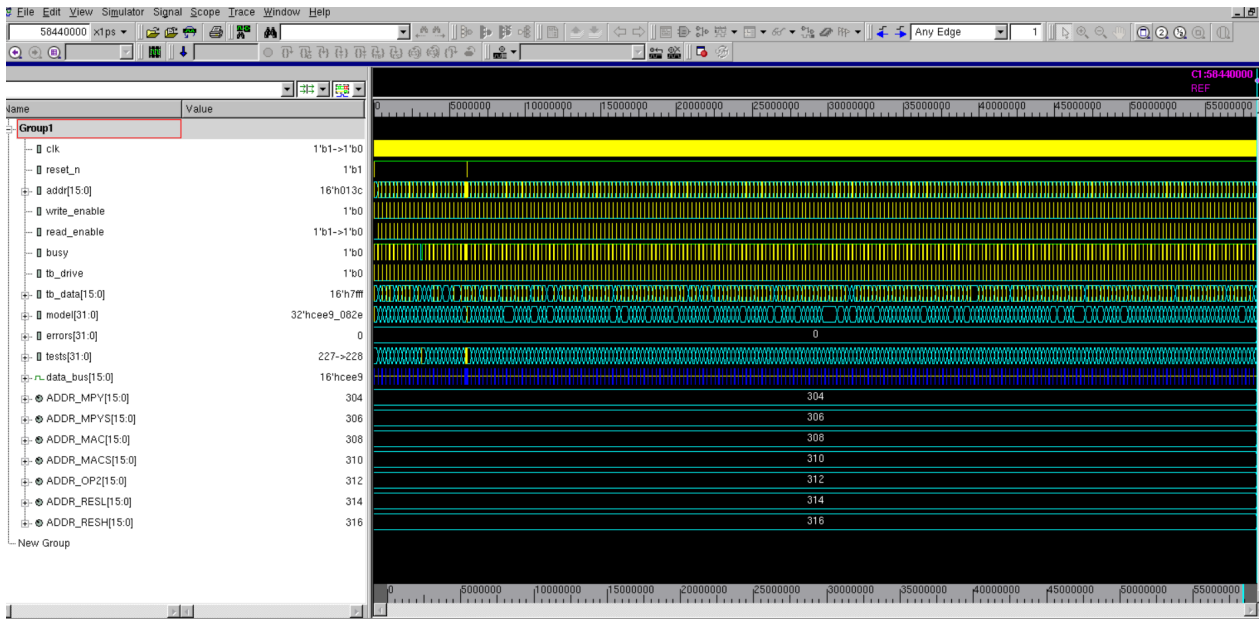


Figure 5: The full run: tests reaches 228 and errors stays 0.

4 Synthesis

Tri-state fix

The TA warned that Design Compiler may build the tri-state bus from BSLEX cells, which ignore the enable and act like inverters (3 + 5 would read as 247). Blocking them leaves the correct TNBUFFX cells:

```
set_dont_use [get_lib_cells */BSLEX1]
set_dont_use [get_lib_cells */BSLEX2]
set_dont_use [get_lib_cells */BSLEX4]
```

Getting timing to pass

	5 ns	5 ns, speed effort	7 ns (final)
Slack	-1.29 VIOLATED	-1.37 VIOLATED	0.00 MET
Violating paths	19	17	0
Critical path	6.21 ns	6.29 ns	6.91 ns
Cells	998	1019	881
Design area	14542.17	14666.44	13462.00

Table 6: The 32-bit ripple-carry adder needs about 6.3 ns, so 5 ns could not pass. The spec gives no clock speed, so I used 7 ns.

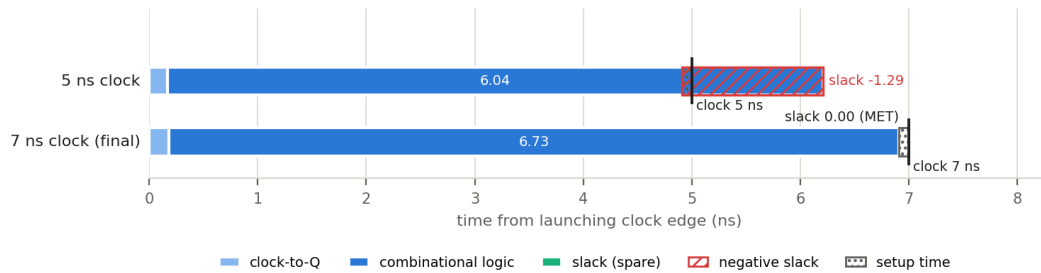


Figure 6: Critical-path timing at 5 ns and 7 ns.

At 7 ns the path is slightly *longer* (6.91 ns) but still passes. With `set_max_area 0`, DC uses spare time to pick smaller, slower gates, which also cut the area by about 1080 units.

Combinational area	7521.18
Flip-flop area	5156.35
Net area	784.47
Total	13462.00

Table 7: Area at 7 ns.

Critical path

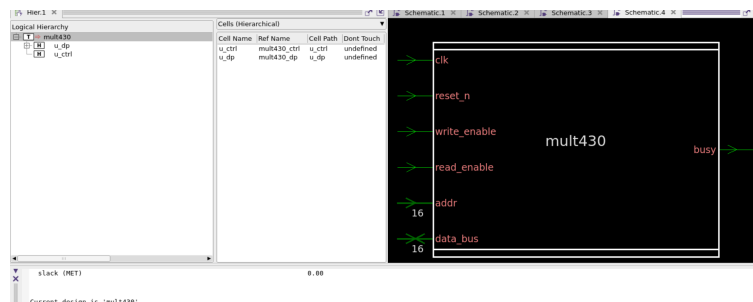
`u_dp/mcand_reg[0]` → 32-bit ripple-carry adder (product + mcand) → `u_dp/product_reg[31]`. 53 logic levels, 6.91 ns, slack 0.00 ns at 7 ns (≈ 143 MHz).

5 Gate-level check

```
grep -c BSLEX output/assignment3_synth.v      # 0
grep -c TNBUFF output/assignment3_synth.v     # 16 (one per data_bus bit)
./ps_sim/assignment3_synth.simv               # 228 tests, 0 errors
```

The netlist passes the same 228 tests with the same random split. The bus-release tests passing on real gates shows the tri-state fix worked.

6 Design Vision

Figure 7: The synthesized block has exactly the ports in the spec; `data_bus` shows the two-way arrow of an inout.

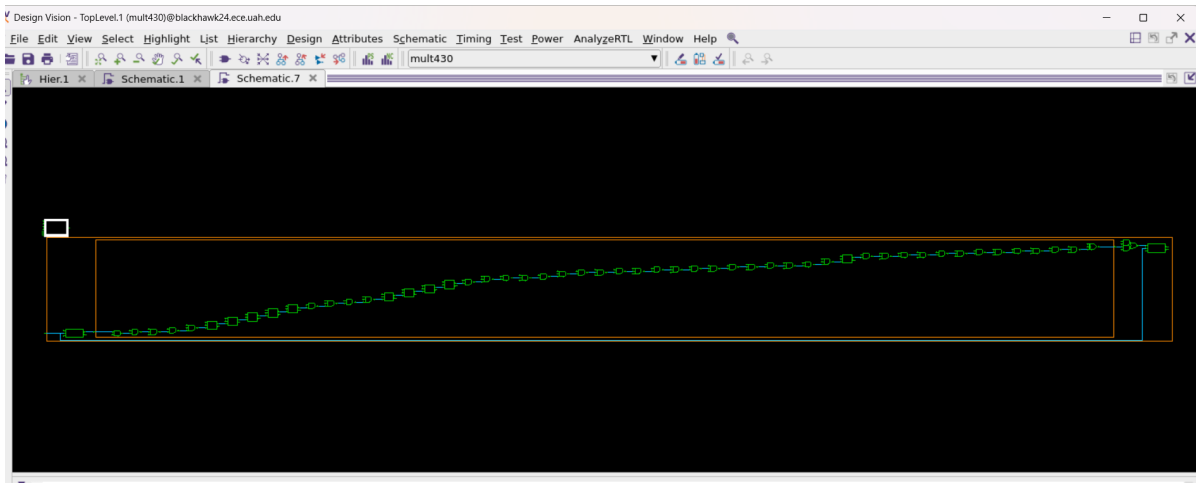


Figure 8: The critical path: the staircase is the 32-bit carry chain.

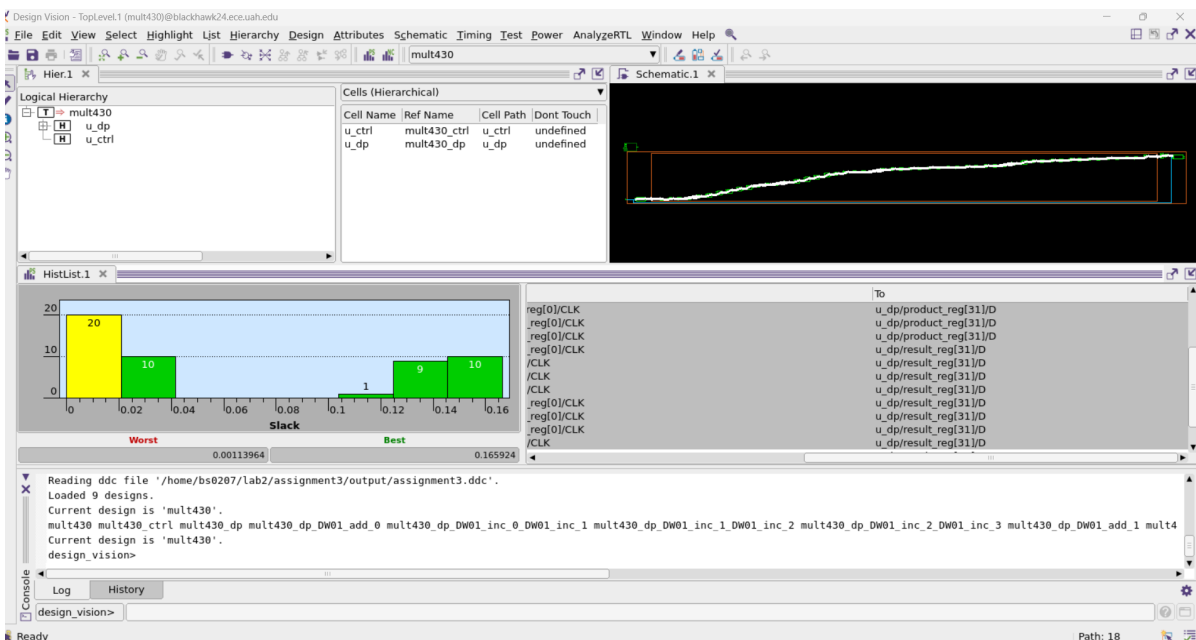


Figure 9: Slack of the 50 worst paths: none negative. The worst ones end at product_reg[31] and result_reg[31].

7 Choices I made

Clock	7 ns, because 5 ns failed and the spec sets no speed
Writes during busy	Ignored, so an operation cannot be corrupted
Accumulator	One shared 32-bit result; MPY/MPYS overwrite, MAC/MACS add
Overflow	Wraps at 32 bits (no overflow flag in the spec)
Data bus	Real inout with the BSLEX fix

8 What I learned

- One unsigned multiplier plus a sign fix covers all four modes.

- A wide adder sets the clock speed: 32-bit ripple carry limited the design to about 143 MHz. Adding 16 bits per step instead would be the next improvement.
- A test is only useful if it can fail: the write-while-busy test writes to the MAC address for that reason.
- Always check the netlist for known bad cells (BSLEX) and re-run the full testbench on the gates.

A Source files

RTL: src/assignment3.sv

```

/*****
MSP430 Hardware Multiplier Accelerator
Modes: MPY (0x0130), MPYS (0x0132), MAC (0x0134), MACS (0x0136)
OP2 write (0x0138) starts the operation
Result: RESL (0x013A) = result[15:0], RESH (0x013C) = result[31:16]
Multiply method: 16-cycle shift-and-add on magnitudes, sign fixed at end
*****/

//=====
// Control unit: FSM IDLE -> LOAD -> CALC (16 cycles) -> DONE
//=====

`timescale 1ns/1ps

module mult430_ctrl (
    input logic clk,
    input logic reset_n,
    input logic start,          // pulse: OP2 was written
    output logic load,          // load magnitudes into datapath
    output logic step,          // do one shift-and-add step
    output logic finish,        // apply sign, update result
    output logic busy
);
    typedef enum logic [1:0] {IDLE = 2'b00, LOAD = 2'b01,
                             CALC = 2'b10, DONE = 2'b11} state_t;
    state_t state, next_state;
    logic [3:0] count;

    // state register
    always_ff @(posedge clk or negedge reset_n)
        if (!reset_n) state <= IDLE;
        else state <= next_state;

    // step counter (counts 0..15 while in CALC)
    always_ff @(posedge clk or negedge reset_n)
        if (!reset_n) count <= 4'd0;
        else if (state == CALC) count <= count + 4'd1;
        else count <= 4'd0;

    // next-state logic
    always_comb begin
        next_state = state;
        case (state)
            IDLE: if (start) next_state = LOAD;
            LOAD: next_state = CALC;
            CALC: if (count == 4'd15) next_state = DONE;
            DONE: next_state = IDLE;
        endcase
    end

```

```

        endcase
    end

    // outputs
    assign load    = (state == LOAD);
    assign step    = (state == CALC);
    assign finish  = (state == DONE);
    assign busy    = (state != IDLE);
endmodule

//=====
// Datapath: magnitudes, shift-and-add, sign fix, accumulate
//=====
module mult430_dp (
    input logic      clk,
    input logic      reset_n,
    input logic [15:0] op1,
    input logic [15:0] op2,
    input logic [1:0] mode,    // mode[0] = signed, mode[1] = accumulate
    input logic      load,
    input logic      step,
    input logic      finish,
    output logic [31:0] result
);
    logic [31:0] mcand;        // multiplicand, shifts left each step
    logic [15:0] mplier;       // multiplier, shifts right each step
    logic [31:0] product;      // running unsigned product
    logic      neg;           // final product must be negated
    logic [15:0] mag1, mag2;   // operand magnitudes
    logic [31:0] signed_prod;  // product with sign applied

    // magnitude = two's complement negate if signed mode and negative
    assign mag1 = (mode[0] && op1[15]) ? (~op1 + 16'd1) : op1;
    assign mag2 = (mode[0] && op2[15]) ? (~op2 + 16'd1) : op2;

    // apply sign at the end
    assign signed_prod = neg ? (~product + 32'd1) : product;

    always_ff @(posedge clk or negedge reset_n)
        if (!reset_n) begin
            mcand    <= 32'd0;
            mplier    <= 16'd0;
            product   <= 32'd0;
            neg       <= 1'b0;
            result    <= 32'd0;
        end
        else if (load) begin
            mcand    <= {16'd0, mag1};
            mplier    <= mag2;
            product   <= 32'd0;
            neg       <= mode[0] & (op1[15] ^ op2[15]);
        end
        else if (step) begin
            if (mplier[0]) product <= product + mcand;
            mcand    <= mcand << 1;
            mplier    <= mplier >> 1;
        end
        else if (finish) begin
            if (mode[1]) result <= result + signed_prod;    // MAC / MACS
            else        result <= signed_prod;             // MPY / MPYS
        end
    end
endmodule

//=====
// Top module: bus interface, registers, tri-state data bus

```

```
//=====
module mult430 (
    input logic      clk,
    input logic      reset_n,
    input logic [15:0] addr,
    inout wire [15:0] data_bus,
    input logic      write_enable,
    input logic      read_enable,
    output logic      busy
);
    localparam logic [15:0] ADDR_MPY  = 16'h0130;
    localparam logic [15:0] ADDR_MPYS = 16'h0132;
    localparam logic [15:0] ADDR_MAC  = 16'h0134;
    localparam logic [15:0] ADDR_MACS = 16'h0136;
    localparam logic [15:0] ADDR_OP2  = 16'h0138;
    localparam logic [15:0] ADDR_RESL  = 16'h013A;
    localparam logic [15:0] ADDR_RESR  = 16'h013C;

    logic [15:0] op1, op2;
    logic [1:0] mode;
    logic [31:0] result;
    logic      wr_ok, start, load, step, finish;
    logic      rd_en;
    logic [15:0] rd_data;

    // a write is accepted only when not busy and not reading
    assign wr_ok = write_enable && !read_enable && !busy;
    assign start = wr_ok && (addr == ADDR_OP2);

    // write registers
    always_ff @(posedge clk or negedge reset_n)
        if (!reset_n) begin
            op1 <= 16'd0;
            op2 <= 16'd0;
            mode <= 2'b00;
        end
        else if (wr_ok) begin
            case (addr)
                ADDR_MPY:  begin op1 <= data_bus; mode <= 2'b00; end
                ADDR_MPYS: begin op1 <= data_bus; mode <= 2'b01; end
                ADDR_MAC:  begin op1 <= data_bus; mode <= 2'b10; end
                ADDR_MACS: begin op1 <= data_bus; mode <= 2'b11; end
                ADDR_OP2:  op2 <= data_bus;
                default:    ;
            endcase
        end

    // read mux
    always_comb begin
        rd_en = 1'b0;
        rd_data = 16'h0000;
        if (read_enable && !write_enable) begin
            if (addr == ADDR_RESL) begin
                rd_en = 1'b1;
                rd_data = result[15:0];
            end
            else if (addr == ADDR_RESR) begin
                rd_en = 1'b1;
                rd_data = result[31:16];
            end
        end
    end

    // tri-state driver: drive only on a valid read, else release the bus
    assign data_bus = rd_en ? rd_data : 16'bz;
endmodule
```

```

// control unit
mult430_ctrl u_ctrl (
    .clk      (clk),
    .reset_n  (reset_n),
    .start    (start),
    .load     (load),
    .step     (step),
    .finish   (finish),
    .busy     (busy)
);

// datapath
mult430_dp u_dp (
    .clk      (clk),
    .reset_n  (reset_n),
    .op1      (op1),
    .op2      (op2),
    .mode     (mode),
    .load     (load),
    .step     (step),
    .finish   (finish),
    .result   (result)
);
endmodule

```

Testbench: src/assignment3_tb.sv

```

/*****
Testbench for MSP430 Hardware Multiplier (mult430)
The testbench acts as the CPU:
    cpu_write(addr, data) -> drives data_bus, pulses write_enable
    cpu_read (addr, data) -> releases data_bus, pulses read_enable
A software model computes the expected result for every test.
*****/

`timescale 1ns/1ps

module mult430_tb;

    // address map
    localparam logic [15:0] ADDR_MPY  = 16'h0130;
    localparam logic [15:0] ADDR_MPYS = 16'h0132;
    localparam logic [15:0] ADDR_MAC  = 16'h0134;
    localparam logic [15:0] ADDR_MACS = 16'h0136;
    localparam logic [15:0] ADDR_OP2  = 16'h0138;
    localparam logic [15:0] ADDR_RESL  = 16'h013A;
    localparam logic [15:0] ADDR_RESR  = 16'h013C;

    // DUT signals
    logic      clk;
    logic      reset_n;
    logic [15:0] addr;
    wire [15:0] data_bus;
    logic      write_enable;
    logic      read_enable;
    logic      busy;

    // CPU side tri-state driver
    logic      tb_drive;

```

```

logic [15:0] tb_data;
assign data_bus = tb_drive ? tb_data : 16'bz;

// bookkeeping
logic [31:0] model;           // expected contents of the result register
integer      errors;
integer      tests;

// DUT
mult430 dut (
    .clk          (clk),
    .reset_n      (reset_n),
    .addr         (addr),
    .data_bus     (data_bus),
    .write_enable (write_enable),
    .read_enable  (read_enable),
    .busy         (busy)
);

// clock: 10 ns period
always #5 clk = ~clk;

//-----
// CPU write: set signals on negedge, DUT captures on posedge
//-----
task automatic cpu_write(input logic [15:0] a, input logic [15:0] d);
    @(negedge clk);
    addr      = a;
    tb_data   = d;
    tb_drive  = 1'b1;
    write_enable = 1'b1;
    @(negedge clk);
    write_enable = 1'b0;
    tb_drive    = 1'b0;
endtask

//-----
// CPU read: release bus, sample half a cycle later
//-----
task automatic cpu_read(input logic [15:0] a, output logic [15:0] d);
    @(negedge clk);
    addr      = a;
    tb_drive  = 1'b0;
    read_enable = 1'b1;
    @(posedge clk);
    d = data_bus;
    @(negedge clk);
    read_enable = 1'b0;
endtask

//-----
// wait until the multiplier is no longer busy
//-----
task automatic wait_done();
    @(negedge clk);
    while (busy) @(negedge clk);
endtask

//-----
// expected product for a given mode
//-----
function automatic logic [31:0] expected_product(input logic [1:0] m,
                                                  input logic [15:0] a,
                                                  input logic [15:0] b);

    logic signed [31:0] sa, sb;

```

```

    if (m[0]) begin
        sa = $signed(a);           // sign-extend to 32 bits
        sb = $signed(b);
        expected_product = sa * sb;
    end
    else
        expected_product = {16'd0, a} * {16'd0, b};
endfunction

//-----
// run one operation and check RESL / RESH
//-----
task automatic run_op(input string name, input logic [15:0] mode_addr,
                     input logic [15:0] a, input logic [15:0] b);

    logic [1:0] m;
    logic [15:0] lo, hi;
    logic [31:0] got;

    m = mode_addr[2:1];           // 0130->00, 0132->01, 0134->10, 0136->11
    if (m[1]) model = model + expected_product(m, a, b);
    else
        model = expected_product(m, a, b);

    cpu_write(mode_addr, a);
    cpu_write(ADDR_OP2, b);
    wait_done();
    cpu_read(ADDR_RESL, lo);
    cpu_read(ADDR_RESB, hi);
    got = {hi, lo};

    tests = tests + 1;
    if (got == model)
        $display("PASS %-5s a=0x%h b=0x%h result=0x%h", name, a, b, got);
    else begin
        $display("FAIL %-5s a=0x%h b=0x%h result=0x%h expected=0x%h",
                 name, a, b, got, model);
        errors = errors + 1;
    end
endtask

//-----
// test sequence
//-----
initial begin
    clk      = 1'b0;
    reset_n  = 1'b0;
    addr     = 16'h0000;
    write_enable = 1'b0;
    read_enable  = 1'b0;
    tb_drive    = 1'b0;
    tb_data     = 16'h0000;
    model       = 32'd0;
    errors      = 0;
    tests      = 0;

    repeat (2) @(posedge clk);
    @(negedge clk);
    reset_n = 1'b1;

    $display("\n--- Unsigned multiply (MPY) ---");
    run_op("MPY", ADDR_MPY, 16'd3, 16'd5);           // 15
    run_op("MPY", ADDR_MPY, 16'hFFFF, 16'hFFFF);     // 0xFFFFE001
    run_op("MPY", ADDR_MPY, 16'd1234, 16'd0);        // 0

    $display("\n--- Signed multiply (MPYS) ---");
    run_op("MPYS", ADDR_MPYS, 16'hFFFD, 16'd5);     // -3 * 5 = -15

```

```

run_op("MPYS", ADDR_MPYS, 16'hFFFC, 16'hFFFA); // -4 * -6 = 24
run_op("MPYS", ADDR_MPYS, 16'h8000, 16'h8000); // -32768^2 = 0x40000000
run_op("MPYS", ADDR_MPYS, 16'd0, 16'hFFFF); // clear result to 0

$display("\n---- Unsigned multiply-accumulate (MAC) ----");
run_op("MAC", ADDR_MAC, 16'd2, 16'd10); // 0 + 20 = 20
run_op("MAC", ADDR_MAC, 16'd4, 16'd10); // 20 + 40 = 60

$display("\n---- Signed multiply-accumulate (MACS) ----");
run_op("MACS", ADDR_MACS, 16'hFFFE, 16'd10); // 60 - 20 = 40
run_op("MACS", ADDR_MACS, 16'hFFFB, 16'hFFFB); // 40 + 25 = 65

$display("\n---- Write while busy must be ignored ----");
model = expected_product(2'b00, 16'd7, 16'd7); // expect 49
cpu_write(ADDR_MPY, 16'd7);
cpu_write(ADDR_OP2, 16'd7);
cpu_write(ADDR_MAC, 16'd100); // MAC write while busy -> must be
ignored
wait_done();
begin
    logic [15:0] lo, hi;
    cpu_read(ADDR_RESL, lo);
    cpu_read(ADDR_RESH, hi);
    tests = tests + 1;
    if ({hi, lo} === model)
        $display("PASS BUSY result=0x%h", {hi, lo});
    else begin
        $display("FAIL BUSY result=0x%h expected=0x%h", {hi, lo}, model);
        errors = errors + 1;
    end
end

$display("\n---- Bus released when not reading ----");
begin
    logic [15:0] v;
    // 1) idle: nobody should drive the bus
    @(negedge clk);
    tests = tests + 1;
    if (data_bus === 16'hzzzz)
        $display("PASS BUSZ idle bus=0x%h", data_bus);
    else begin
        $display("FAIL BUSZ idle bus=0x%h expected=zzzz", data_bus);
        errors = errors + 1;
    end
    // 2) read a non-result address: multiplier must not drive
    cpu_read(ADDR_OP2, v);
    tests = tests + 1;
    if (v === 16'hzzzz)
        $display("PASS BUSZ read 0x0138 bus=0x%h", v);
    else begin
        $display("FAIL BUSZ read 0x0138 bus=0x%h expected=zzzz", v);
        errors = errors + 1;
    end
end

$display("\n---- Mode switching ----");
run_op("MAC", ADDR_MAC, 16'd3, 16'd3); // 49 + 9 = 0x0000003A
run_op("MPYS", ADDR_MPYS, 16'hFFFF, 16'hFFFF); // -1 * -1 = 0x00000001
run_op("MPY", ADDR_MPY, 16'hFFFF, 16'd2); // 65535 * 2 = 0x0001FFFE
run_op("MACS", ADDR_MACS, 16'hFFFF, 16'd3); // 0x1FFFE - 3 = 0x0001FFFB
run_op("MAC", ADDR_MAC, 16'hFFFF, 16'd1); // + 65535 = 0x0002FFFA

$display("\n---- Accumulate overflow / wrap-around ----");
run_op("MPY", ADDR_MPY, 16'hFFFF, 16'hFFFF); // 0xFFFE0001
run_op("MAC", ADDR_MAC, 16'hFFFF, 16'hFFFF); // 0x1_FFFC0002 -> 0xFFFC0002

```

```

run_op("MPYS", ADDR_MPYS, 16'd0,      16'd0);      // clear to 0
run_op("MACS", ADDR_MACS, 16'hFFFF, 16'd1);      // 0 - 1 -> 0xFFFFFFFF
run_op("MAC",  ADDR_MAC,  16'd1,      16'd1);      // 0xFFFFFFFF + 1 -> 0x00000000

$display("\n--- Reset in the middle of an operation ---");
run_op("MPY",  ADDR_MPY,  16'h1234, 16'h0010);      // non-zero first: 0x00012340
begin
    logic [15:0] lo, hi;
    logic        busy_before;
    cpu_write(ADDR_MPY, 16'd7);
    cpu_write(ADDR_OP2, 16'd9);                      // start 7 * 9
    repeat (5) @(negedge clk);                      // let it run a few steps
    busy_before = busy;
    reset_n = 1'b0;                                  // async reset, no clock edge
    #1;
    tests = tests + 1;
    if (busy_before === 1'b1 && busy === 1'b0)
        $display("PASS RST  busy was 1, dropped to 0 immediately on reset");
    else begin
        $display("FAIL RST  busy before=%b after=%b (expected 1 then 0)",
            busy_before, busy);
        errors = errors + 1;
    end
    repeat (2) @(negedge clk);
    reset_n = 1'b1;                                  // release reset
    model = 32'd0;                                    // reset clears the result
    cpu_read(ADDR_RESL, lo);
    cpu_read(ADDR_RESR, hi);
    tests = tests + 1;
    if ({hi, lo} === model)
        $display("PASS RST  result cleared to 0x'h", {hi, lo});
    else begin
        $display("FAIL RST  result=0x'h expected=0x'h", {hi, lo}, model);
        errors = errors + 1;
    end
end
end
run_op("MPY",  ADDR_MPY,  16'd6,      16'd7);      // works after reset: 0x0000002A

$display("\n--- Random tests (200 operations, all modes, seed 430) ---");
begin
    logic [15:0] ra, rb, lo, hi, maddr;
    logic [1:0]  m;
    integer      i, rand_err;
    integer      cnt_mpy, cnt_mpys, cnt_mac, cnt_macs;
    void'($urandom(430));                            // fixed seed -> repeatable
    rand_err = 0;
    cnt_mpy = 0; cnt_mpys = 0; cnt_mac = 0; cnt_macs = 0;
    for (i = 0; i < 200; i = i + 1) begin
        // random mode: 0130, 0132, 0134, 0136
        m      = $urandom_range(0, 3);
        maddr = ADDR_MPY + {13'd0, m, 1'b0};
        case (m)
            2'b00: cnt_mpy  = cnt_mpy  + 1;
            2'b01: cnt_mpys = cnt_mpys + 1;
            2'b10: cnt_mac  = cnt_mac  + 1;
            2'b11: cnt_macs = cnt_macs + 1;
        endcase
        // operands, biased toward edge values
        case ($urandom_range(0, 7))
            0: ra = 16'h0000; 1: ra = 16'hFFFF;
            2: ra = 16'h8000; 3: ra = 16'h7FFF;
            default: ra = $urandom;
        endcase
    end
end

```

```

        case ($urandom_range(0, 7))
            0: rb = 16'h0000;  1: rb = 16'hFFFF;
            2: rb = 16'h8000;  3: rb = 16'h7FFF;
            default: rb = $urandom;
        endcase
        // expected value
        if (m[1]) model = model + expected_product(m, ra, rb);
        else      model = expected_product(m, ra, rb);
        // run on the DUT
        cpu_write(maddr,  ra);
        cpu_write(ADDR_OP2, rb);
        wait_done();
        cpu_read(ADDR_RES_L, lo);
        cpu_read(ADDR_RES_H, hi);
        tests = tests + 1;
        if ({hi, lo} != model) begin
            rand_err = rand_err + 1;
            errors   = errors + 1;
            if (rand_err <= 10)
                $display("FAIL  RAND  #%0d mode=%b a=0x%h b=0x%h result=0x%h expected=0x%h"
,
                                i, m, ra, rb, {hi, lo}, model);
        end
    end
    $display("RAND  200 ops (MPY %0d, MPYS %0d, MAC %0d, MACS %0d): %0d failures",
        cnt_mpy, cnt_mpys, cnt_mac, cnt_macs, rand_err);
end

$display("\n=== %0d tests, %0d errors ===", tests, errors);
if (errors == 0) $display("All tests completed successfully\n");
$finish;
end

// waveform dump
initial begin
    $dumpfile("assignment3.dump");
    $dumpvars(0, mult430_tb);
end

endmodule

```

Synthesis script: src/syn.tcl

```

# =====
# DC synthesis script: Assignment 3 (MSP430 hardware multiplier)
# Run from inside assignment3/ :  dc_shell -f syn.tcl
# =====

# ---- clock period in ns (change this to experiment) ----
set CLK_PERIOD 7

# ---- search path and work library ----
lappend search_path src/
define_design_lib WORK -path work

# ---- technology libraries ----
set LIB_DIR /apps/designlib/SAED90_EDK/SAED_EDK90nm/Digital_Standard_cell_Library/synopsys/models
set link_library [list $LIB_DIR/saed90nm_max.db $LIB_DIR/saed90nm_typ.db $LIB_DIR/saed90nm_min.db]

```

```
set target_library [list $LIB_DIR/saed90nm_typ.db]

# ---- TA fix: never use BSLEX cells for tri-states ----
set_dont_use [get_lib_cells */BSLEX1]
set_dont_use [get_lib_cells */BSLEX2]
set_dont_use [get_lib_cells */BSLEX4]

# ---- load design ----
analyze -library WORK -format sverilog assignment3.sv
elaborate -architecture verilog -library WORK mult430
check_design > reports/synth_check_design.rpt

# ---- constraints ----
create_clock clk -name ideal_clock1 -period $CLK_PERIOD
set_input_delay 2.0 [remove_from_collection [all_inputs] clk]
set_output_delay 2.0 [all_outputs]
set_max_area 0

# ---- synthesize ----
compile -map_effort medium -area_effort medium

# ---- reports ----
report_area > reports/assignment3_area.rpt
report_timing > reports/assignment3_timing.rpt
report_resources > reports/assignment3_resources.rpt
report_constraints > reports/assignment3_constraints.rpt
report_qor > reports/assignment3_qor.rpt

# ---- outputs ----
write_sdc constraints/assignment3.sdc
write -f ddc -hierarchy -output output/assignment3.ddc
write -hierarchy -format verilog -output output/assignment3_synth.v

# ---- show critical path, open Design Vision ----
report_timing
gui_start
```